

Model verification (and validation introduction)

Modeling 2025 lecture notes
Sasha D. Hafner

[See also: Thackner et al. 2024.](#)

Concepts of Model Verification and Validation

Ben H. Thacker*

Scott W. Doebbling

Francois M. Hemez

Mark C. Anderson

Jason E. Pepin

Edward A. Rodriguez

LA-14167-MS

Issued: October 2004

Plan for today:

12:15-12:25 Course evaluation review

12:25-12:45 Model verification background

12:45-13:00 Model verification demo

13:00-13:10 Break

13:10-14:00 Demo, discussion, and exercise on verification

Midterm evaluation summary

- * You are learning things
- * Videos are helpful
- * Exercises and clear solutions are helpful
- * My writing is too messy
- * Sometimes too much too quickly
- * Time to work on exercises
- * Inconsistent advice

"Model verification and validation (V&V) is an enabling methodology for the development of computational models that can be used to make engineering predictions with quantified confidence"

Thackner et al., 2024

Programming errors are not uncommon.
Finding them after a model has been applied is quite uncomfortable.
Better to do careful verification!

"Verification is the process of determining that a model implementation accurately represents the developer's conceptual description of the model and its solution. Validation is the process of determining the degree to which a model is an accurate representation of the real world from the perspective of the intended uses of the model."

There are many different ways to screw up! Verification checks one way and validation another on top of verification.

"V&V cannot prove that a model is correct and accurate for all possible scenarios, but, rather, it can provide evidence that the model is sufficiently accurate for its intended use."

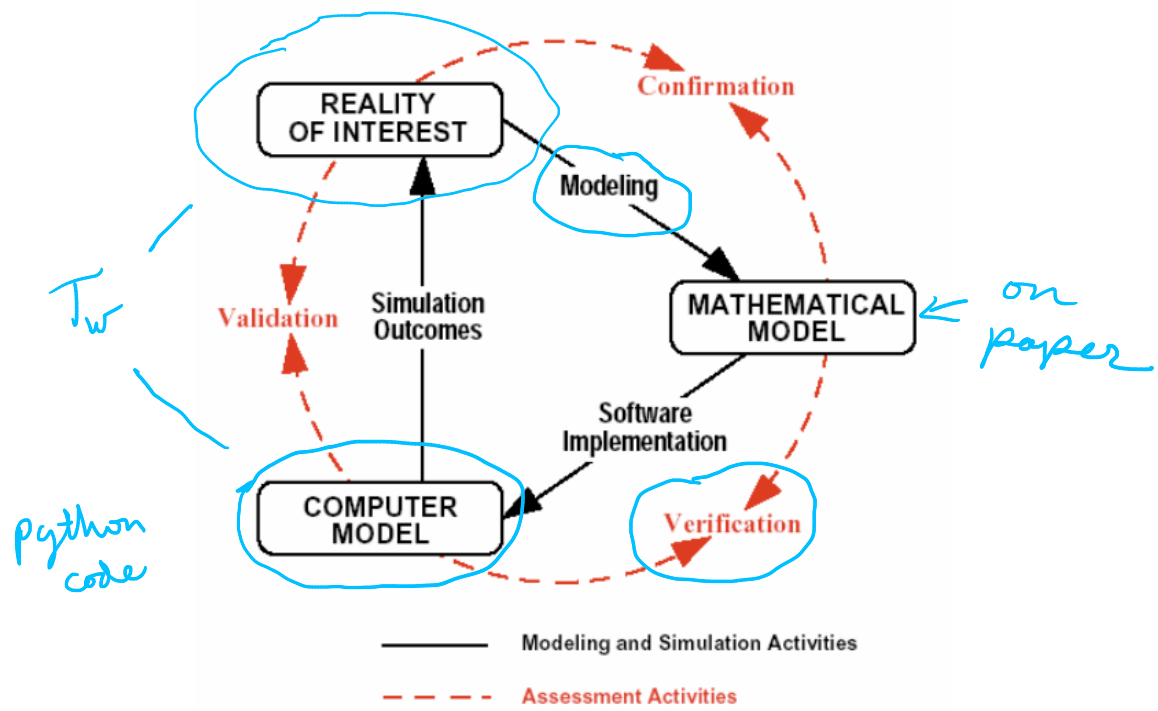


Figure 1: Simplified view of the model verification and validation process. [8]

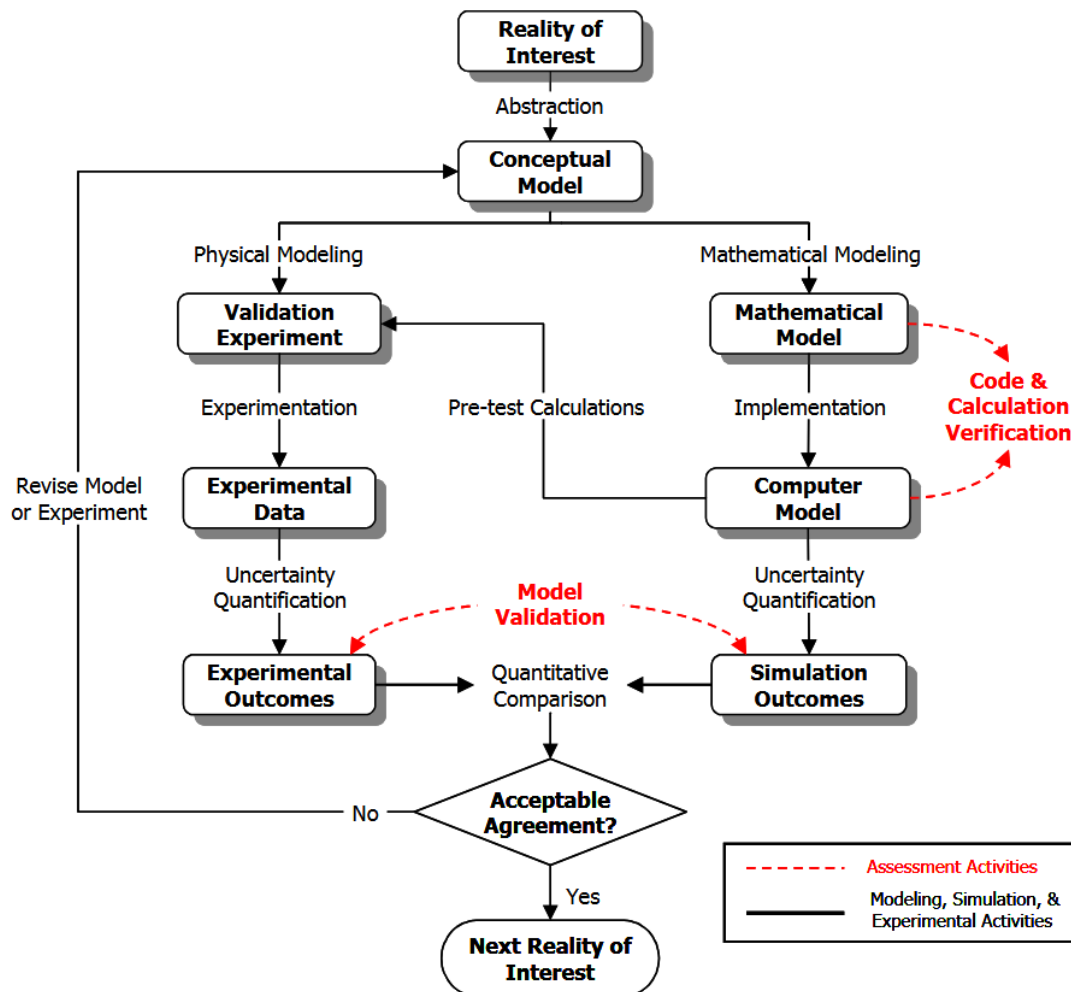


Figure 2: Detailed model development, verification, and validation process.

Model verification

Verification means checking that you have implemented a model correctly

You should check these:

- * Conservation (e.g., mass or energy),
- * constitutive equations, and
- * ultimate or limiting cases. — *where you know what to expect*

Exactly what can be done and how to do it varies among models.

Some verification can be done with careful calls to a model function and analysis of the results. The idea is to see if the model does what the mathematical equations used for its formulation say it should do.

With this approach you can find problems without information on the causes.

Then experience, model call "experiments", and a check of the code can help identify the causes.

Some will require digging into the model code itself from the start, and perhaps exporting intermediate results for checking. Even better, you can explicitly add cumulative transport as a state variable.

When checking code, you should look for equations completeness, correctness, and unit consistency.

Warning: Terminology is not used consistently among disciplines! Interpretation differs and you may also see "evaluation".

Model verification requires a good understanding of the conceptual model! Otherwise, how would you know what to expect?

*+ mathematical
model*

Model validation

Model validation is intended to be a check of the model itself, not the software implementation per se. So it should be done after model verification. Otherwise we won't know whether a poor fit is a problem with the model or its implementation (the software).

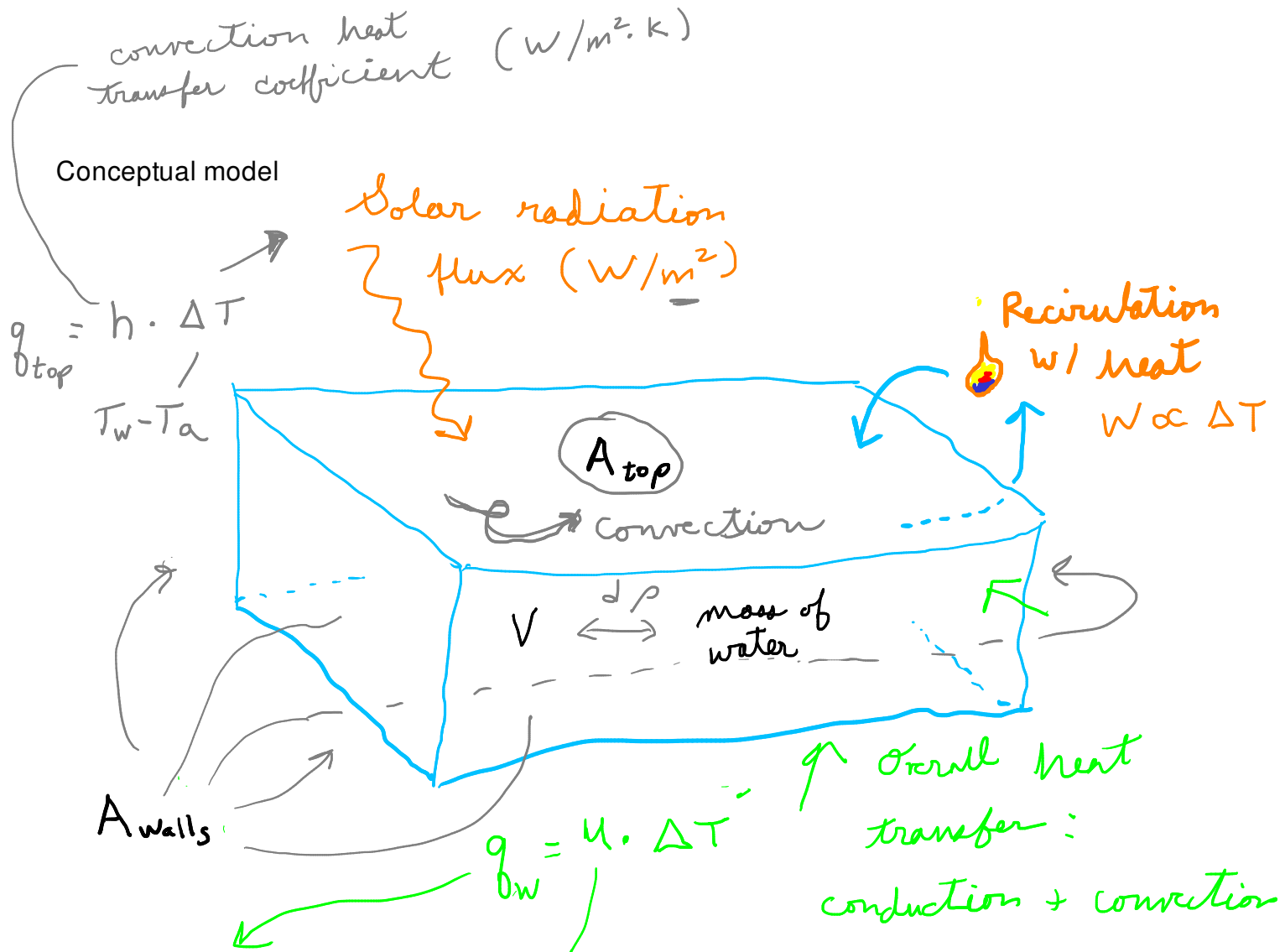
Validation is usually done by comparing model predictions to actual measurements.

We will work on validation later, probably Thursday.

Pool demo

Verification approaches

- * Be sure to understand conceptual model
- * Compare code to mathematical conservation equation
- * Check and document code units
- * Expect steady-state or equilibrium? Does value make sense?
- * Compare to closed-form solution if available
- * Check limiting cases
- * Get into function with breakpoint() if needed

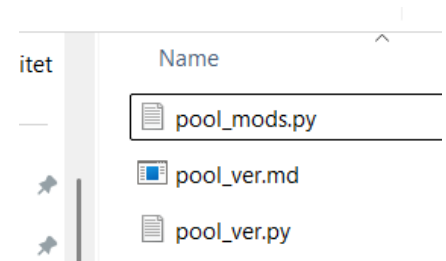


We use "U" for overall heat transfer coefficient, which is analogous to h ($\text{W}/\text{m}^2 \cdot \text{K}$). See class 3 slides / notes for more details (p. 13).

model function is defined in a module.

```
# Define model function
def dynmod(a_top,
           a_wall,
           depth,
           q_sol,
           u_top,
           u_wall,
           temp_air,
           temp_sub,
           flow_renew,
           temp_renew,
           temp_init,
           times,
           cp = 4180,
           dens = 1000
           ):

    """
    Dynamic model for heat loss fr
```



Watch out!

Two common problems:

1. Be sure that the module file name matches the import or reload call.
2. Working directory needs to be the directory that has the module file. In Spyder use F5 to fix that.

Derivative function is defined inside model function

```
# Define rates function
def rates(t, temp_pool):

    # Net energy, expressed as positive for increase
    # One nice things about numerical solutions is you are not required to
    # In fact it can be helpful to understand the different pathways to k

    # Calculate heat flow (W) through individual pathways
    # All are positive for heat flow in (this is arbitrary)
    Q_solar = a_top * q_sol # Solar en
    Q_top = a_top * u_top * (temp_air - temp_pool) # Convect
    Q_sub = a_wall * u_wall * (temp_sub - temp_pool) # Conduct
    Q_renew = cp * dens * flow_renew * (temp_renew - temp_pool) # Net ene

    # And sum them
    Q_net = Q_solar + Q_top + Q_sub + Q_renew

    # Express as temperature change
    dtemp_dt = Q_net / (cp * dens * vol)

    return dtemp_dt
```

Our module is imported with import pool_mods as pm
reloaded with reload(pm) and called with
pm.dynmod(...)

```
import pool_mods as pm

# Use as needed
# reload(pm)

# Set areas (all m^2).
# Assume pool is 10 x 4 x 2 m.
length = 10
width = 4
depth = 2
a_top = length * width
a_wall = 2 * (length + width) * depth + a_top

pred01 = pm.dynmod(a_top = a_top,
                  a_wall = a_wall,
                  depth = depth,
                  q_sol = 0,
                  u_top = 100,
                  u_wall = 3,
                  temp_air = 10,
                  temp_sub = 10,
                  flow_renew = 0,
                  temp_renew = 0,
                  temp_init = 30,
                  times = np.arange(0, 12 * 3600 + 3600, 3600)
)

pred01

plt.plot(pred01.t / 3600, pred01.y[0, :])
plt.xlabel('Time (h)')
plt.ylabel('Pool temperature (deg. C)')
plt.savefig('pred01.png')
```

Conservation - heat energy balance

Rate of energy accumulation = solar + air/top + soil/substrate + renewal

All in W

Do we have this in the model code?

```
# And sum them  
Q_net = Q_solar + Q_top + Q_renew
```

First error: missing soil/substrate term

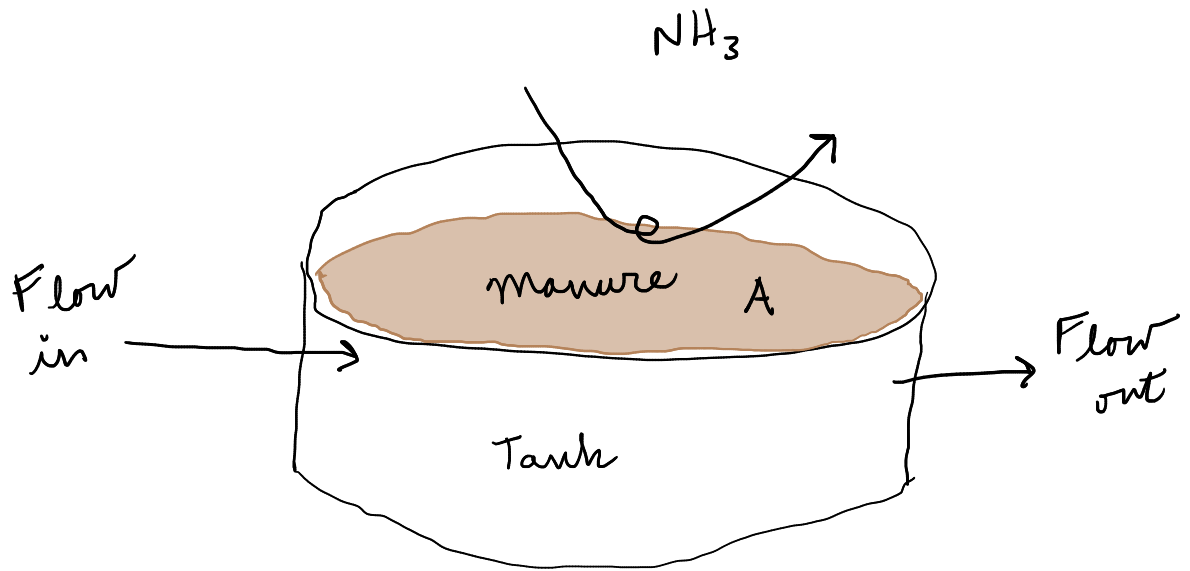
See md file.

See py script.

[All in demos subdirectory.](#)

See py module.

Ammonia conceptual model



See files in exercise directory for more details.